Clase 4 — Conmutación: circuitos, paquetes y mensajes

Bloque: A (Fundamentos) · **Semana:** 2 · **Duración estimada:** 110 min **Correlativa:** Clase 3 (encapsulación y comparación OSI ↔ TCP/IP) · **Sigue con:** Clase 5 (el retardo en las redes de paquetes) **Material de referencia:** Tanenbaum cap. 1 §1.3 (pág. ~10 a ~20 del PDF de tu carpeta) · RFC 9293, RFC 2737, RFC 3209, RFC 3031

🎯 Al terminar esta clase vas a poder

1. **Distinguir** las tres técnicas de conmutación por un único criterio: **cuándo se toma la decisión** (al establecer la comunicación, o por cada unidad de datos).
2. **Dibujar** el establecimiento de un circuito virtual en la red telefónica y explicar qué recursos quedan reservados.
3. **Explicar** el store-and-forward y **por qué un router no puede reenviar bit a bit sin un buffer infinito**.
4. **Calcular** el retardo de un mensaje de 10 000 bits dividido en 4 paquetes de 2 500 bits que atraviesa 3 enlaces, con la aritmética a la vista, y **compararlo** con el mensaje entero y con el circuito virtual.
5. **Construir** la tabla comparativa de las tres técnicas con: conexión, tamaño de la unidad, reserva de recursos, retardo, posibilidad de reenrutar, orden de llegada y error típico.
6. **Explicar por qué los paquetes de tamaño variable no se usan**: *unfairness* y retardo extremo.
7. **Resolver enteros** el Examen `2007-12 P3` (10 pts) y el Examen `2009-02 P4` (9 pts).

> **Prerequisito del examen:** `2007-12 P3` (10 pts — las tres técnicas con análisis comparativo), `2009-02 P4` (9 pts — datagramas vs. circuitos virtuales), `2011-03 P5` (10 pts — retardo con store-and-forward), `2009-07 P2` (10 pts — distancia entre hops como métrica).

PARTE 1 — TEORÍA

1.1 El criterio que las distingue a todas

Hay tres técnicas de conmutación. Se distinguen por **una sola pregunta**:

```
┌──────────────────────────────────────────────────────────────┐
│  ¿LA DECISIÓN DE ENCAMINAR SE TOMA ANTES DE EMPEZAR A MANDAR │
│  DATOS, O SE TOMA PARA CADA UNIDAD DE DATOS?                 │
│                                                              │
│  ANTES  ──►  conmutación por CIRCUITOS   (y circuitos virtuales) │
│  CADA UNO ─►  conmutación por PAQUETES    (datagramas)       │
│  CADA UNO ─►  conmutación por MENSAJES    (el antecesor)     │
└──────────────────────────────────────────────────────────────┘
```

Todo lo demás se deriva de esa respuesta:

Pregunta	Circuitos	Paquetes	Mensajes
¿Cuándo se decide?	**Antes**, una vez	Por cada paquete	Por cada mensaje
¿Hay conexión?	**Sí**	No (datagramas)	No
¿Se reservan recursos?	**Sí**	No	No
¿El retardo es?	**Constante**	Variable	Variable y grande
¿El camino es el mismo para todos los paquetes?	**Sí**	Puede cambiar	Puede cambiar

Y una observación de fondo que vale para las tres: las tres son **estrategias para compartir un medio de transmisión** entre varias comunicaciones. La diferencia no es técnica de hardware: es **cuándo se decide el reparto del medio**.

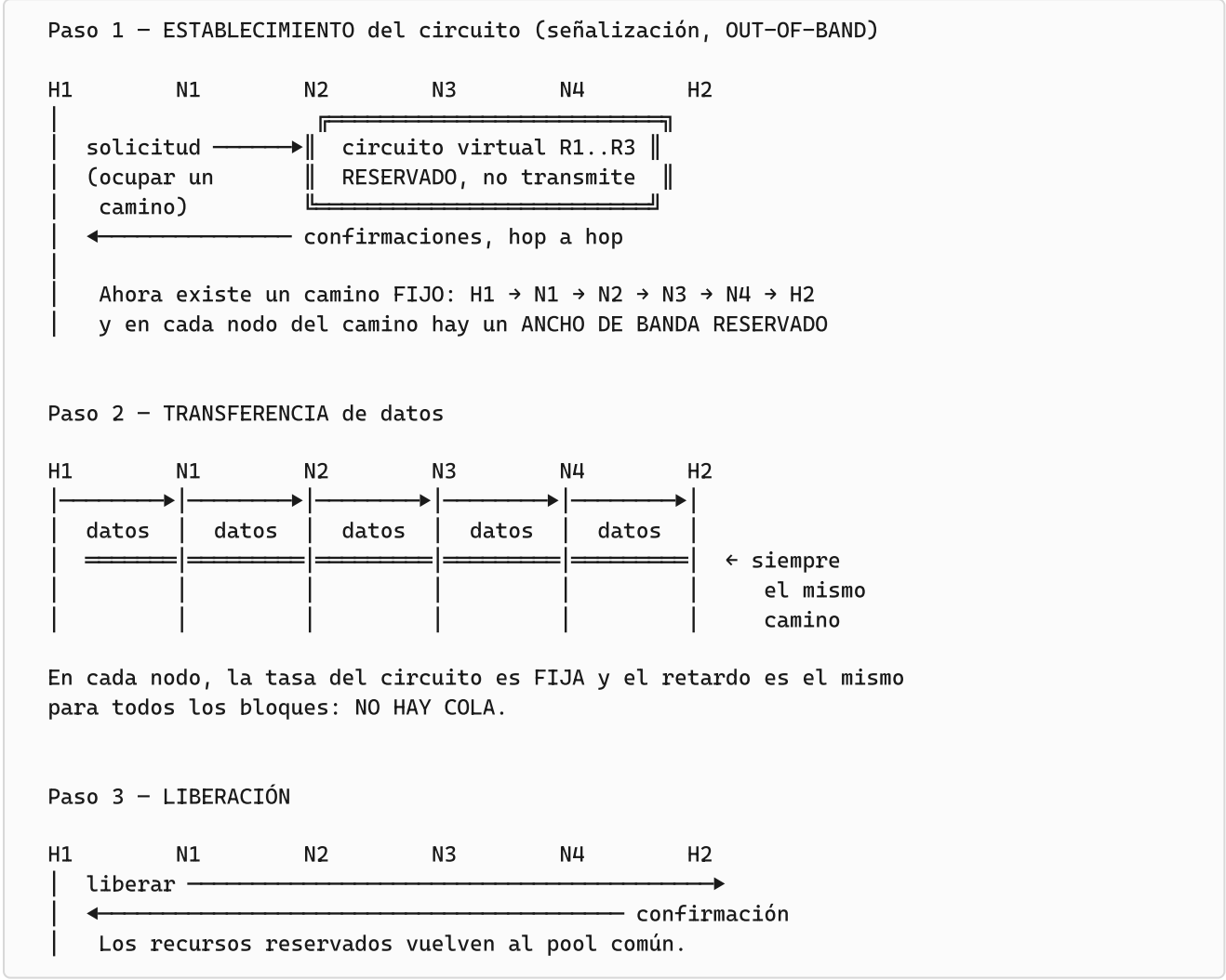
```
MEDIO COMPARTIDO:  ═══════════════════════════════════
La pregunta es: ¿cómo le doy el medio a cada uno?

CIRCUITOS:  "A vos te tocan estos 64 kbps, para siempre.
             Los demás no te los tocan."        → reparto FIJO, static
PAQUETES:   "El que tenga algo para mandar manda,
             y si el medio está libre lo usa."    → reparto DINÁMICO
MENSAJES:   "Solo cuando tengo el mensaje entero
             lo transmito entero."                → reparto DINÁMICO,
                                                    pero con bloqueos largos
```

1.2 Conmutación por circuitos (*circuit switching*)

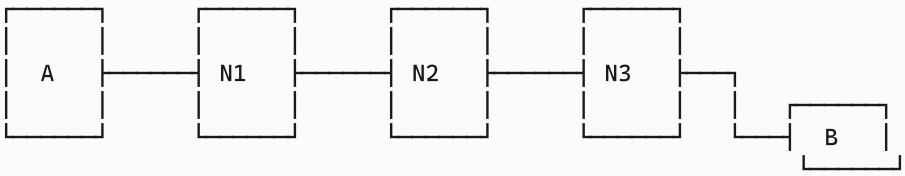
Cómo funciona

Se **establece un circuito** antes de poder mandar datos, y ese circuito **reserva recursos** en cada nodo intermedio.



El ejemplo canónico: la red telefónica

La red telefónica clásica (POTS, *Plain Old Telephone Service*) es la implementación más limpia de conmutación por circuitos, y **todavía lo es**:



A llama a B. La señalización engancha un circuito de 4 kHz en CADA enlace del camino. Los 4 nodats intermedios reservan su ancho de banda para ese circuito. Si un nodo del camino se rompe, la llamada SE CAE. No hay ruta alternativa: el circuito es un camino físico (o lógico) fijo.

Por qué la red telefónica es de circuitos y no de paquetes:

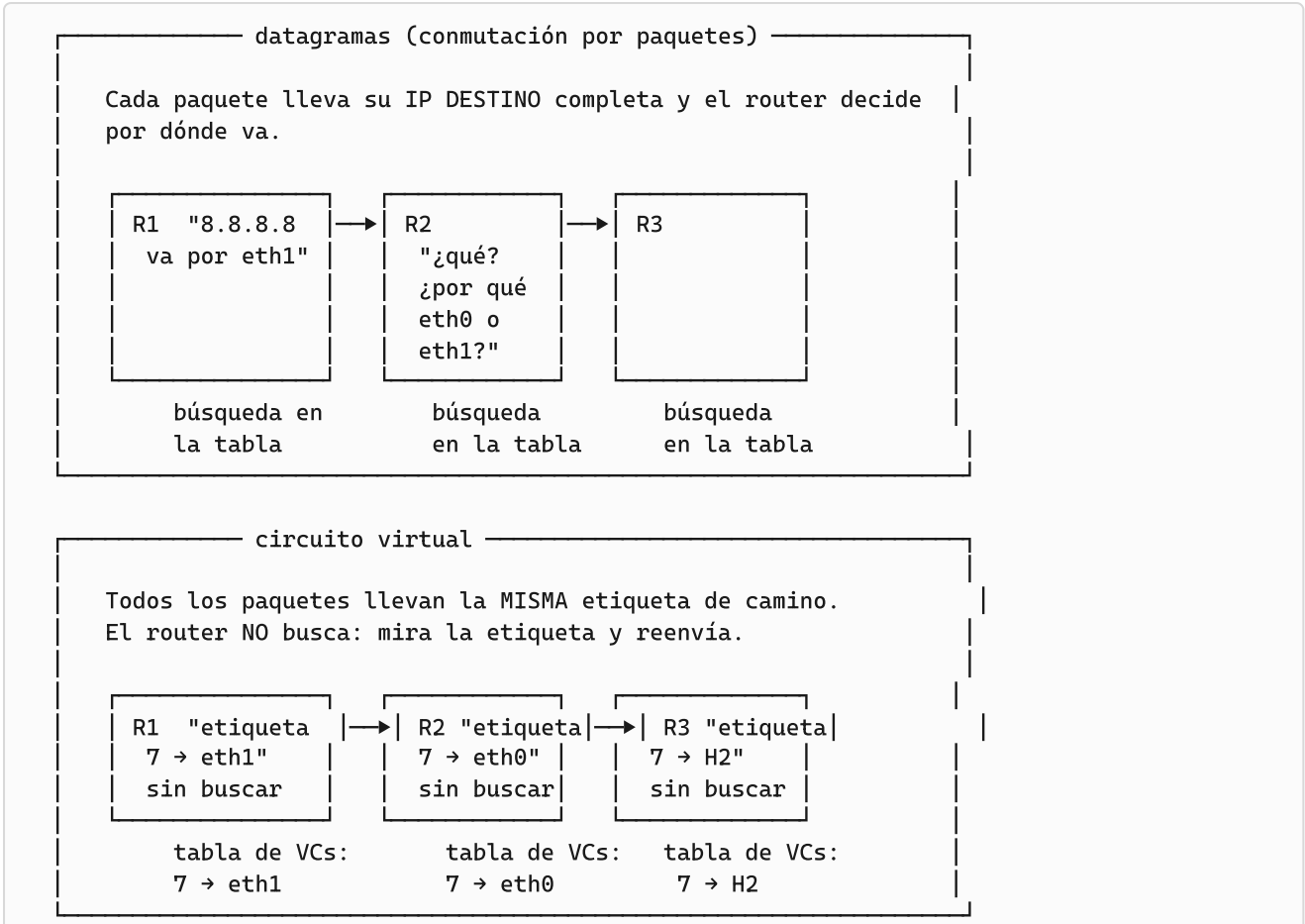
- **La voz necesita retardo constante.** Si un paquete de voz llegara en 20 ms y el siguiente en 200 ms, la conversación sería ininteligible. Un circuito reservado garantiza **retardo constante**, que es lo que hace posible el servicios Telefónico clásico.
- **La voz necesita un ancho de banda constante.** Un circuito de 64 kbps reservados da exactamente 64 kbps siempre. Con paquetes, la voz compite con todo lo demás.
- **La relación de precios es fija.** La voz se cobraba por minuto, y el circuito reservado es lo que hace que el minuto sea medible.

Y las consecuencias negativas, que son las que hacen que Internet no use circuitos:

- **El retardo de establecer el circuito.** Establecerlo lleva de 100 a 500 ms (o hasta 2 s con señalización SS7 y la búsqueda de ruta). Si vas a transferir 300 bytes, no vale la pena.
- **Si se cae un nodo, se rompe todo.** No hay ruta alternativa.
- **Los recursos quedan ociosos.** Si hablás 10 minutos con pausas de un minuto entre frases, el circuito reservado está pagado pero sin usar.
- **No escala.** El número de circuitos simultáneos entre dos nodos es **finito** y hay que dimensionarlo por anticipado.

Circuitos virtuales: la versión "de paquetes" de los circuitos

Circuito virtual (*virtual circuit*, VC) es el híbrido: **se establece un camino lógico antes de mandar los datos**, pero **no se reservan recursos físicos**. Cada paquete lleva una **etiqueta** que identifica el camino, y los routers reenvían **según la etiqueta**, sin consultar la IP de destino.



Diferencias clave respecto del circuito clásico:

	Circuito (físico)	Circuito virtual
Se reserva ancho de banda	**Sí**, en cada nodo	**No** (salvo QoS)
El retardo	**Constante**	**Variable**: depende de la cola
Si se cae un nodo	Se rompe todo	Se rompe el circuito: hay que **reestablecer**
El router busca por	El cable físico (no busca)	**La etiqueta** (busca, pero en tabla chica)
Coste de establecer	Alto (se ocupa el medio)	**Bajo** (solo un mensaje de señalización)

Dónde vive el circuito virtual hoy: Frame Relay, **ATM**, y sobre todo **MPLS**. En MPLS, el "circuito virtual" es el **LSP** (*Label Switched Path*): un túnel unidireccional que se identifica por una **etiqueta** de 20 bits apilable. El router MPLS **no consulta la IP de destino**: consulta la etiqueta, que es una entrada en un array. Eso es **más rápido** y es la razón por la que MPLS se usa en el backbone de los operadores: la decisión es un índice, no un longest prefix match. (Clase 45 lo desarrolla entero.)

Y el dato de 2026: el plano de control de MPLS **se está moviendo a Segment Routing** (`RFC 8402`, `RFC 8660`, y `RFC 8986` **SRv6**), que **elimina el MPLS del plano de datos en el caso de SRv6**: las etiquetas se codifican en la cabecera IPv6 (el *Segment Routing Header*). El circuito virtual sigue existiendo conceptualmente, pero la implementación es otra.

1.3 Conmutación por mensajes (*message switching*)

Cómo funciona

Cada **mensaje completo** (tamaño arbitrario) es reenviado de nodo en nodo. El nodo **lo recibe entero**, **lo guarda en un buffer**, y **espera a poder transmitirlo completo** por el siguiente enlace. Es **store-and-forward sin fragmentar**.

```
Un mensaje de 8000 bits, 4 nodos intermedios, enlaces de 1000 bit/s:

H1 ─── N1 ─── N2 ─── N3 ─── H2

t=0        t=8      t=16     t=24     t=32     t=40   (segundos)

|─8000b─▶|          |          |          |          |
|          |─8000b─▶|          |          |          |
|          |          |─8000b─▶|          |          |
|          |          |          |─8000b─▶|          |
|          |          |          |          |─8000b─▶|
|          |          |          |          |          |

Cada salto tarda 8000/1000 = 8 segundos.
El mensaje entero tarda 4 × 8 = 32 segundos en salir del último nodo,
y 40 segundos en llegar a H2.

⚠️ El RETARDO ES ENORME. Y es el mismo para el mensaje entero
  que para cualquier otro: no hay pipeline.
```

El store-and-forward y por qué el router tiene que esperar el paquete entero

Este es el **argumento central** de la clase, y conecta directo con la Clase 5. Hay que entenderlo bien.

El store-and-forward es un compromiso entre memoria y velocidad. La razón por la que un router **no** puede reenviar bit a bit (o byte a byte) es esta:

```
Si el router reenvía bit a bit (cut-through), el PROBLEMA es este:

H1 (100 Mbps) ⟹ N1 ⟹ N2 ⟹ N3 (1 Mbps)

Bit a bit, N1 empieza a mandar hacia N2 CADA VEZ QUE LLEGA UN BIT de H1.

Como H1 manda a 100 Mbps y N1->N2 transmite a 10 Mbps, por cada bit
que N1 transmite tiene que GUARDAR 10 bits que le llegan.

En 1 segundo:
   entran 100 000 000 bits
   salen    10 000 000 bits
   se guardan 90 000 000 bits = 11,25 MB

⚠️ Para no perder NADA, N1 necesitaría un buffer INFINITO.

┌──────────────────────────────────────────────────────────┐
│  Si el buffer es FINITO (y lo es), el exceso se DESCARTA.  │
│  Y un paquete descartado es un paquete perdido para       │
│  siempre, porque IP no recupera de pérdidas.              │
│                                                           │
│  ⇒ CON ELTA DEGRADADO, EL CORTAFALLOS ES PEOR.           │
└──────────────────────────────────────────────────────────┘
```

La solución que eligió Internet: store-and-forward con tamaño de paquete acotado. El trade-off completo:

```
Store-and-forward con paquetes de tamaño FIJO (L):

┌──────────────────────────────────────────────────────────────┐
│  El tamaño del buffer es finito y el tamaño del paquete      │
│  está DISEÑADO para que quepan muchos paquetes completos.    │
│                                                              │
│  Si el paquete mide L bytes, un buffer de B bytes            │
│  almacena ⌊B / L⌋ paquetes completos.                        │
│                                                              │
│  Con L ACOTADO:                                              │
│     · el buffer se llena y se descartan paquetes completos   │
│       (no bites sueltos) → se pierde UNO, no la mitad de uno │
│     · la unidad deLoss es predecible                         │
│     · el router puede reordenar y descartar                  │
│                                                              │
│  El tamaño MÁXIMO del paquete es una decisión de diseño      │
│  que hace que todo esto funcione: MTU = 1500 en Ethernet.    │
└──────────────────────────────────────────────────────────────┘
```

El argumento completo, paso a paso:

1. Un router tiene **buffers finitos**. (Los routers reales tienen entre 4 MB y 512 MB, según la clase.)
2. El medio tiene **velocidades distintas** entre entrada y salida.
3. Si reenvía **bit a bit**, el buffer crece sin límite → o hace falta memoria infinita, o se descarta y se pierde información.
4. Si reenvía **por paquetes completos**, el buffer necesario es `⌊B/L⌋ · L` bytes, que es **acotado y conocido**.

5. Cuando el buffer se llena, se **descarta un paquete entero**. La unidad de pérdida es **un paquete, no un bloque de bytes**.
6. Eso **no elimina** la pérdida, pero la hace **predecible** y permite que las capas superiores la manejen (TCP la recupera; UDP no).
7. **Conclusión: el tamaño máximo del paquete es el precio de la conmutación por paquetes.** Y ese precio se paga en el MTU.

La lista de.waiters: el problema de la conmutación por mensajes

El problema más grave de la conmutación por mensajes no es el retardo: es el **bloqueo de cabeza** (*head-of-line blocking*) y la **injusticia**.

```
N1 tiene UN solo enlace de salida (hacia N2), de 1000 bit/s.
En el buffer hay:

   [  mensaje A: 8000 bits  ]  ← llegó primero, se transmite entero
   [  mensaje B: 20 bits    ]  ← llegó después, NO PUEDE salir
   [  mensaje C: 5000 bits  ]  ← llegó después, NO PUEDE salir

N1 transmite A completo. Durante 8 segundos, B y C esperan.
Después transmite C completo. B espera 5 segundos más.
B llevaba 13 segundos de espera por un mensaje de 20 bits.

⚠️ INJUSTICIA: un mensaje pequeñoDelayed por un mensaje grande
   que estaba delante. En un sistema de tiempo real, B es
   inservible.
```

Por qué no se usa la conmutación por mensajes hoy:

- **Retardo enorme y no predecible.** El retardo depende del tamaño de la cola, que depende de los demás. No podés dar una garantía.
- **Bloqueo de cabeza.** Un mensaje grande retrasa a todos los que están detrás.
- **Requiere buffers enormes.** Para no perder un mensaje de 1 MB hay que tener 1 MB de buffer, y **los routers tienen MB, no GB**. El MTU de un mensaje tendría que ser enorme.
- **El análisis de la Pregunta 5 del Examen 2009-07** (10 pts) muestra por qué esto es un problema de enrutamiento, no solo de conmutación.

Pero tampoco es inútil como idea: es exactamente lo que hacen los **store-and-forward de la capa de aplicación** y los **enrutadores de mensajeros**:

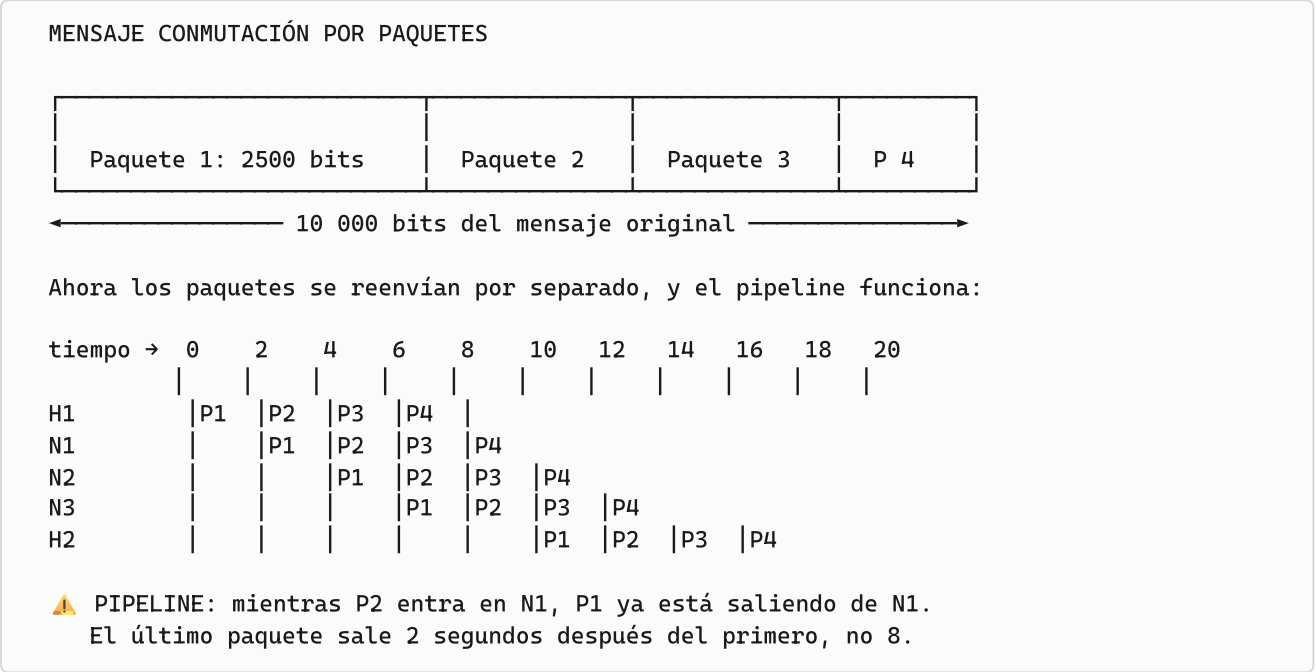
- **Un *mail relay*** (MTA) que recibe un email completo y lo reenvía después.
- **Un router de BGP que recibe un UPDATE completo** y lo procesa.
- **Un forwarder de logging** que recibe un evento completo.
- **La conmutación de mensajes en el contexto de las "SAN"** (*Storage Area Networks*) y los sistemas de archivos distribuidos.

Conclusión histórica: la conmutación por mensajes **es la generalización de las otras dos**. Si el mensaje es del tamaño de un circuito y hay uno reservado, es conmutación de circuitos. Si el mensaje se divide en paquetes acotados, es conmutación por paquetes. **La conmutación por mensajes es el caso sin acotar**, y por eso es la peor de las tres.

1.4 Conmutación por paquetes (*packet switching*)

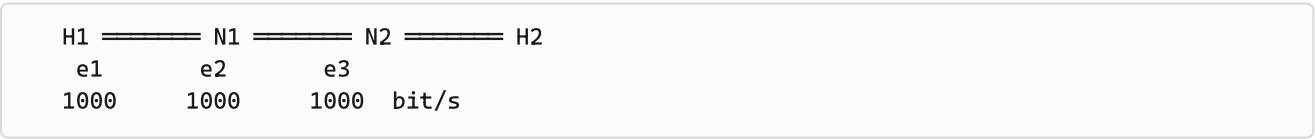
La idea

Los mensajes se **dividen en paquetes de tamaño fijo o acotado**, y cada paquete se reenvía **de forma independiente** con store-and-forward.



El ejemplo numérico completo

Datos: mensaje de **10 000 bits**, dividido en **4 paquetes de 2 500 bits**. **Tres enlaces** de **1 000 bit/s** cada uno, unidos por **dos conmutadores** (N1 y N2). `d_proc` despreciable, sin cola.



Paso 1 — Retardo de transmisión de un paquete en cada enlace.

```
d_trans = L / R = 2500 bits / 1000 bit/s = 2,5 segundos
```

Paso 2 — El comportamiento de cada conmutador (store-and-forward de paquetes).

En cada nodo, **recibo un paquete entero antes de poder empezar a transmitir el siguiente**. Eso significa que el nodo está ocupado `2,5 s` con cada paquete:

```
e1 (H1→N1):  transmite P1 de 0 a 2,5 ; P2 de 2,5 a 5,0 ; P3 de 5,0 a 7,5 ; P4 de 7,5 a 10,0
e2 (N1→N2):  transmite P1 de 2,5 a 5,0 ; P2 de 5,0 a 7,5 ; P3 de 7,5 a 10,0 ; P4 de 10,0 a 1
e3 (N2→H2):  transmite P1 de 5,0 a 7,5 ; P2 de 7,5 a 10,0 ; P3 de 10,0 a 12,5 ; P4 de 12,5 a
```

Paso 3 — El retardo de cada paquete, extremo a extremo.

Cada paquete atraviesa 3 enlaces y pasa por 2 conmutadores (cada uno con `d_proc`):

```
d(P) = d_proc(N1) + L/R(e1) + L/R(e2) + d_proc(N2) + L/R(e3) + Σd_prop
     =      0     +   2,5   +   2,5   +     0      +   2,5   +   Σd_prop
     =       7,5 s  +  Σ d_prop
```

Con `d_prop = 0` (enlaces de distancia despreciable):

```
d(P1) = 7,5 s      llega a H2 en t = 7,5 s
d(P2) = 7,5 s      llega a H2 en t = 10,0 s
d(P3) = 7,5 s      llega a H2 en t = 12,5 s
d(P4) = 7,5 s      llega a H2 en t = 15,0 s
```

Paso 4 — El retardo del mensaje COMPLETO (lo que le importa al usuario que espera el archivo).

El mensaje está completo cuando llega **el último** paquete:

```
d(mensaje) = 15,0 segundos
```

Paso 5 — La comparación con las otras dos técnicas.

Si en lugar de 4 paquetes de 2500 bits:

```
┌──────────────────────┬──────────────────────────────────────────┐
│  TÉCNICA             │  ¿Cuándo está listo el último byte?      │
├──────────────────────┼──────────────────────────────────────────┤
│  MENSAJES            │                                          │
│  (1 mensaje de 10000)│  4 saltos × (10000/1000) = 4 × 10 = 40 s │
│                      │                                          │
│  PAQUETES            │                                          │
│  (4 paquetes de 2500)│  último paquete entra en e3 en t=12,5    │
│                      │  y tarda 2500/1000 = 2,5 s más           │
│                      │  → 15,0 s                                │
│                      │                                          │
│  CIRCUITO            │                                          │
│  (reservado 1000b/s) │  10000/1000 = 10 s, SIN STORE-AND-FORWARD│
│                      │  (el circuito nunca tiene el enlace libre)│
├──────────────────────┼──────────────────────────────────────────┤
│  Ganador             │  CIRCUITO (10 s), pero necesita 3 s de   │
│                      │  establecimiento → 13 s en el mejor caso │
└──────────────────────┴──────────────────────────────────────────┘
```

El análisis completo, con las cuatro cifras a la vista:

Técnica	Establecimiento	Retardo del último byte	**Total**
Mensajes	0	4 × 10 = **40 s**	**40 s**
Paquetes	0	12,5 + 2,5 = **15 s**	**15 s**
Circuito	3 s	10 000/1000 = **10 s**	**13 s**

Interpretación — y acá está el resultado clave:

- **Paquetes (15 s) vs. mensajes (40 s): los paquetes son 2,67 veces más rápidos.** La razón: el **pipeline**. Con un solo mensaje, cada nodo espera a recibirlo entero (10 s) antes de empezar a mandar; con paquetes, cada nodo espera solo 2,5 s y después puede seguir trabajando.
- **Circuito (13 s) es el más rápido**, pero con una condición: **el circuito tiene que existir de antemano**. Si se establece ahora, son 13 s. Y **solo gana si el mensaje es grande**: si es de 2000 bits, el circuito tarda 3 s de establecimiento + 2 s de datos = 5 s, y los paquetes 1 + 2 × 2 = ... calculemos: 1 paquete de 2000 bits, `d = 3 × (2000/1000) = 6 s` → **6 s, más rápido que el circuito**.
- **El punto clave: el circuito gana para mensajes GRANDES y el paquete para mensajes CHICOS.** Y para tráfico intermitente, el circuito es **desastroso**: si A y B se turnan para hablar 1 segundo cada uno, el circuito está reservado el 100 % del tiempo para una conversación que usa el 100 % del tiempo.

La fórmula general del retardo con paquetes

Esto es lo que se pide en la **Pregunta 5 del Examen 2011-03** (10 puntos), y lo resuelvo entero en la Clase 6:

```
Con N paquetes de tamaño L en una línea de n enlaces, todos de
velocidad R, con store-and-forward y sin cola ni propagación:

· El primer paquete tarda:      n · (L/R)     [n saltos de store-and-forward]
· Cada paquete siguiente tarda: (L/R)         [solo el último enlace]
· El último de los N paquetes tarda:
      n·(L/R) + (N-1)·(L/R) = (N + n − 1) · L/R
```

La interpretación del resultado `(N + n − 1)·L/R`:

```
N  = cantidad de paquetes   (el trabajo)
n  = cantidad de enlaces    (el pipeline)

N · L/R   = el trabajo puro: es IGUAL en todas las técnicas
(n-1)·L/R = el costo del pipeline: el primer paquete tiene que
            recorrer los n enlaces uno por uno; cada uno después
            sólo tiene que recorrer el último.

→Mientras más paquetes (N grande), más se amortiza el pipeline
  (n-1)·L/R y más se parece al tiempo del circuito.
→ Mientras más enlaces (n grande), más caro es el pipeline.
```

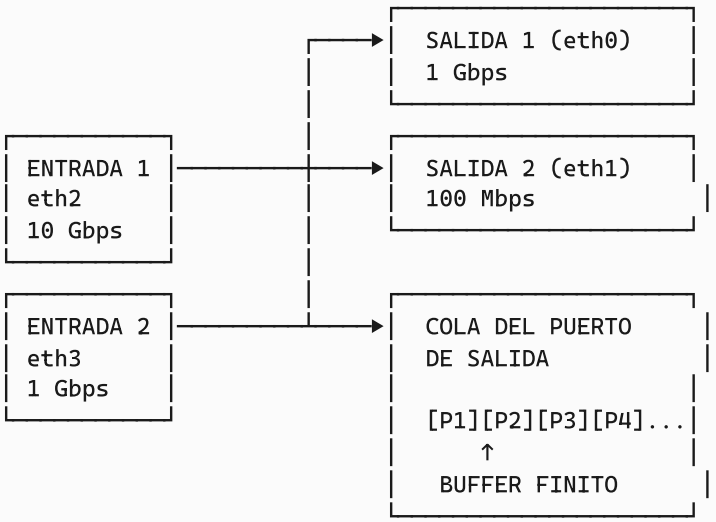
El límite importante:

```
N → ∞   (paquetes infinitamente chicos)
d → n · L/R = 10000/1000 = 10 s   ← el tiempo del circuito

⚠️ PERO: si los paquetes son demasiado chicos, el overhead los
  ahoga. Con 20 B de cabecera por paquete, un mensaje de 10 000
  bits con paquetes de 100 bits tiene 100 paquetes y 2000 bytes
  de overhead: el 17 % del tráfico es cabecera.
```

1.5 El router: 3 entradas, 2 salidas, buffers llenándose

El diagrama que explica por qué existe el retardo de cola, y por qué es el **único componente variable** del retardo.




```
⚠️ eth2 entra a 10 Gbps, pero sale por eth0 (1 Gbps) o
   eth1 (100 Mbps). El 90 % o el 99 % del tráfico que entra
   por eth2 NO SE PUEDE TRANSMITIR en ese instante.

El buffer de la cola de salida de eth1 (100 Mbps) SE LLENA.
Los paquetes que no entran se DESCARTAN.
```

La aritmética del desbordamiento, que es donde nace la pérdida:

```
Supongamos:
   eth2 envía 10 Gbps de paquetes de 1500 bytes
   eth1 puede sacar 100 Mbps

En 1 segundo:
   llegan    10 000 000 000 bits / 1500 bits = 6 666 667 paquetes
   salen        100 000 000 bits / 1500 bits =     66 667 paquetes

   ⚠️ Se acumulan 6 666 667 − 66 667 = 6 600 000 paquetes por segundo
      = 6 600 000 × 1500 bytes = 9 900 000 000 bytes = 9,9 GB por
      SEGUNDO en la cola.

Un buffer de 4 MB se llena en:
   4 000 000 bytes / 9 900 000 000 bytes/s = 0,0004 s

⇒ Con 4 MB de buffer, en 400 MICROSEGUNDOS se empieza a perder.
```

Eso significa tres cosas, y las tres son importantes:

1. **La pérdida es inevitable** si la tasa de llegada supera la de salida. No es un defecto del router: es aritmética.
2. **El retardo de cola se dispara**: un paquete que llega "encima" de la cola espera **millones de paquetes × su propio tiempo de transmisión**.

3. **El tamaño del buffer NO protege**: agrandarlo 100 veces retrasa la muerte 100 veces, no la evita. Y **agrandar el buffer aumenta el retardo** de los paquetes que sí pasan (bufferbloat, Clase 21).

Y este es el error típico del ejercicio 2.1: dibujar "un router con 3 paquetes en el buffer y todo bien". Lo correcto es:

```
El retardo de cola de un paquete que llega en la posición k de la cola:

    d_cola = k · L / R_salida

Con 6 600 000 paquetes de cola:
    d_cola = 6 600 000 × 1500 bits / 100 000 000 bit/s
           = 6 600 000 × 15 µs
           = 99 segundos    ⚠️ ¡El paquete tarda más de un minuto
                             en salir!

Con un buffer de 4 MB (2666 paquetes de 1500 B):
    capacidad del buffer = 4 194 304 / 1500 = 2796 paquetes
    d_cola máximo = 2796 × 1500 / 100 000 000 = 41,9 ms
    lo que NO entre → se descarta
```

El dato estructural que hay que recordar: el buffer es **una cola de espera**, y como toda cola, tiene un límite. Los routers reales no pueden tener buffers del tamaño del tráfico que deben absorber: si el tráfico es de 100 Gbps y el buffer es de 100 GB, no lo construís. **El buffer siempre es más chico que la carga de la ráfaga.** Por eso la pérdida es parte del diseño normal de Internet, y no una excepción.

1.6 La tabla comparativa de las tres técnicas

Esta tabla es **la respuesta al Examen 2007-12 P3** y hay que memorizarla completa.

Característica	**Conmutación por circuitos**	**Conmutación por paquetes**	**Conmutación por mensajes**
¿Hay conexión?	**Sí**, se establece un circuito	**No** (datagramas) / **sí** (circuito virtual)	No
Cuándo se decide el camino	**Antes**, al establecer	**Por cada paquete**	Por cada mensaje
Unidad de datos	**Flujo continuo** (bytes)	**Paquete** de tamaño fijo o acotado	**Mensaje** completo, tamaño arbitrario
Tamaño típico	El que se reserve (64 kbps telefónicos)	**1500 bytes** (MTU de Ethernet)	**Ilimitado** (MB)
¿Se reservan recursos?	**Sí**, ancho de banda en cada nodo	**No** (mejor esfuerzo)	No
Retardo	**Constante**, sin cola	**Variable**, depende de la cola	**Variable y grande**
¿Se puede reenrutar?	**No**: si cae un nodo, se rompe	**Sí**: cada paquete puede ir por donde sea	Sí
Orden de llegada	**Siempre en orden** (mismo camino)	**Puede llegar desordenado** (caminos distintos)	Puede llegar desordenado
Retardo de establecer	**Alto** (100 ms a 2 s)	**Cero**	Cero
Uso del medio	**Ineficiente** si hay pausas (se paga y no se usa)	**Eficiente**: multiplexado estadístico	Eficiente en teoría, malo en la práctica
Bloqueo de cabeza	No	**Sí** si un paquete grande retrasa a los chicos	**Muy sí**: un mensaje enorme retrasa todo
Tamanjos de buffer	No necesita (recursos reservados)	**Necesita**: $\lfloor B/L \rfloor \cdot L$	**Necesita B ≥ tamaño del mensaje mayor**
Error típico	"Reservar recursos es eficiente"	"El retardo es constante"	"Es lo mismo que paquetes pero más grande"
Dónde se usa	Red telefónica clásica; **MPLS** (LSP)	**Internet**. Frame Relay, ATM	**Prácticamente nada** como tal

La frase resumen de cada una

- **Circuitos:** "*decidí una vez y pagás por el canal aunque no lo uses*".
- **Paquetes:** "*no decido nada por adelantado, mando lo que puedo y me arreglo*".
- **Mensajes:** "*espero a tenerlo todo entero antes de mandar nada*".

Por qué Internet usa paquetes

Cuatro razones, y cada una es un **compromiso que rechaza** una ventaja de los circuitos:

1. **No hay que establecer nada.** El retardo de establecimiento de un circuito (100 ms a 2 s) es inaceptable para una petición web. Con paquetes, la primera petición sale inmediatamente.

2. **No se reserva capacidad ociosa.** El tráfico de Internet es en ráfagas. Reservar 1 Gbps para una conexión que transmite 100 kbps el 10 % del tiempo es tirar dinero. Los paquetes **multiplexan estadísticamente**: los picos de unos llenan los valles de otros.
3. **Se puede reenrutar.** Si cae un router, el siguiente paquete puede ir por otro camino. Un circuito no puede: se rompe. Esta es la diferencia entre **IP** (best effort, sin conexión) y un protocolo orientado a conexión como ATM.
4. **Es stateless en el núcleo.** Un router de Internet **no guarda información de las conexiones**: solo mira la IP destino y reenvía. Puede atender millones de flujos simultáneos con una memoria modesta. Es la razón por la que **IntServ (RFC 2211) no escaló** y DiffServ ganó (Clase 45): mantener estado por flujo en cada router es insostenible.

El precio que se paga por elegir paquetes:

```
┌──────────────────────────────────────────────────────────────┐
│ 1. Retardo variable e impredecible.  Para una llamada de voz │
│    150 ms es inaceptable; para una descarga de archivo da igual.│
│ 2. Pérdidas.  Si se pierde un paquete, se retransmite entero,  │
│    y se duplicó el retardo.                                  │
│ 3. Desorden.  Paquetes por caminos distintos llegan desordenados.│
│ 4. Holgura obligatoria (headroom).  Hay que dejar el medio   │
│    sobrecargado para poder absorber ráfagas.                 │
│    Y sobrecargar significa aumentar el retardo.              │
│    Ese es el dilema del "90 % de utilización es óptimo".     │
└──────────────────────────────────────────────────────────────┘
```

⚠️ Actualización a 2026

Sigue igual — de memoria, sin dudar: - **Las tres técnicas y el criterio de decisión** (antes / por cada unidad de datos). - **Que Internet usa conmutación por paquetes** y que es *best effort*. - **El store-and-forward** y la razón de que el router espere el paquete entero (buffers finitos). - **Que el tamaño máximo del paquete (MTU) es el precio del store-and-forward**. - **La fórmula** `(N + n − 1)·L/R` para el retardo del último paquete. - **Que MPLS implementa circuitos virtuales (LSP) en el plano de datos.**

Cambió o se modificado:

- **El "servicio de circuitos" de la red telephone clásica ya no existe como tal.** La PSTN se cerró en la mayoría de los países y la voz.mouse migró a **VoIP sobre IP**, que es conmutación de paquetes con **un circuito virtual lógico arriba**: los operadores de VoIP **reservan recursos con SIP/IMS** para dar QoS a la voz, que es exactamente "conmutación por circuitos dentro de una red de paquetes". ⇒ **La frontera del Bloque A se corrió: la voz ya no es el ejemplo de circuitos.**
- **El circuito virtual sigue, pero cambió de implementación. MPLS** sigue desplegándose masivamente en los carriers (`RFC 3031` arquitectura, `RFC 3032` pila de etiquetas, `RFC 4364` MPLS VPN). Pero el **plano de control se está moviendo a Segment Routing**: `RFC 8402` (arquitectura), `RFC 8660` (SR-MPLS), `RFC 8986` **(SRv6)**. En **SRv6 el MPLS desaparece del plano de datos**: los segmentos se codifican en un *Segment Routing Header* de IPv6.
- **En el datacenter, MPLS fue reemplazado por EVPN-VXLAN.** `RFC 7432` (EVPN) + `RFC 7348` (VXLAN, que encapsula sobre **UDP/4789**). Es el estándar de overlay de datacenter, no el MPLS de los operadores. Y VXLAN **sí es un circuito virtual**, pero de capa 2, no de capa 3.

- **La conmutación por "mensajes" tiene un caso de uso nuevo: los servicios de *store-and-forward* de capa de aplicación.** Webhooks, colas de mensajes (SQS, Kafka, RabbitMQ), y los **MTA de correo** siguen siendo exactamente conmutación por mensajes: reciben el mensaje completo, lo bufferizan, y lo reenvían cuando pueden.
- **El control de congestión terminó de asumir que la red es de paquetes. CUBIC (`RFC 9438`, 2023, *Standards Track*)** es el default global; en 2012 se enseñaba Reno con "quizá CUBIC". Y **BBR nunca fue RFC**: es `draft-ietf-ccwg-bbr`, *Experimental*, y especifica **BBRv3**.
- **La multicast IP es conmutación por paquetes con State.** IP multicast (`RFC 1112` y siguientes) mantiene **estado por grupo** en los routers que están en el camino, lo cual es un punto medio entre datagramas y circuitos virtuales: la decisión se toma una vez al unirse al grupo, pero **no se reserva ancho de banda**.

> **Consecuencia para el examen:** el **esquema conceptual de las tres técnicas** es de 1981 (Tanenbaum lo teach desde el capítulo 1) y **no cambió**. Lo que cambió es **quién lo usa**: la voz dejó de ser el ejemplo de circuitos, y los circuitos virtuales pasaron de MPLS a SRv6 a VXLAN. Si te preguntan "*¿qué se usa hoy?*", respondé con el esquema y después una línea de actualización.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Los buffers de un router con tres paquetes de llegada simultánea

Enunciado. Un router tiene **3 interfaces de entrada** y **2 de salida**:

- eth2 (entrada): 10 Gbps, por donde entra una ráfaga de **3 paquetes de 1500 bytes** en el mismo instante.
- eth3 (entrada): 1 Gbps, sin tráfico.
- eth0 (salida): 1 Gbps, buffer de **100 000 bytes**.
- eth1 (salida): 100 Mbps, buffer de **20 000 bytes**.

Los tres paquetes están dirigidos a destinos que salen por **eth1**.

a) ¿Cuántos paquetes entran en el buffer de eth1? ¿Cuántos se descartan? **b)** ¿Cuál es el retardo de cola del primer paquete y del último que entra? **c)** ¿Qué pasa si en lugar de 3 paquetes de 1500 bytes entran 3 paquetes de 40 bytes?

Resolución.

Paso 1 — La tasa de llegada y la de salida.

```
eth1 transmite a 100 Mbps.
Llegan 3 paquetes de 1500 bytes = 4500 bytes = 36 000 bits.

Tiempo que tardan los 3 en entrar:
    36 000 bits / 10 000 000 000 bit/s = 0,0000036 s = 3,6 µs

Tiempo que tarda eth1 en sacar los 3:
    36 000 bits / 100 000 000 bit/s = 0,00036 s = 360 µs
```

Interpretación: los 3 paquetes entraron en 3,6 µs y eth1 necesita 360 µs para sacarlos. **El buffer tiene que absorber la diferencia.**

Paso 2 — La capacidad del buffer.

```
Buffer de eth1 = 20 000 bytes
Tamaño de un paquete = 1500 bytes
Capacidad = ⌊20 000 / 1 500⌋ = ⌊13,33⌋ = 13 paquetes

⚠️ OJO: se guardan 13 paquetes COMPLETOS = 13 × 1500 = 19 500 bytes.
   Los 500 bytes restantes no se usan, porque un paquete
   a medias no sirve: o entra entero o se descarta.
   ESTA es la consecuencia dePacketizing: el buffer se usa
   de a paquetes completos.
```

Paso 3 — Respuesta a a).

```
Llegan 3 paquetes. El buffer tiene capacidad para 13.
Los 3 caben.

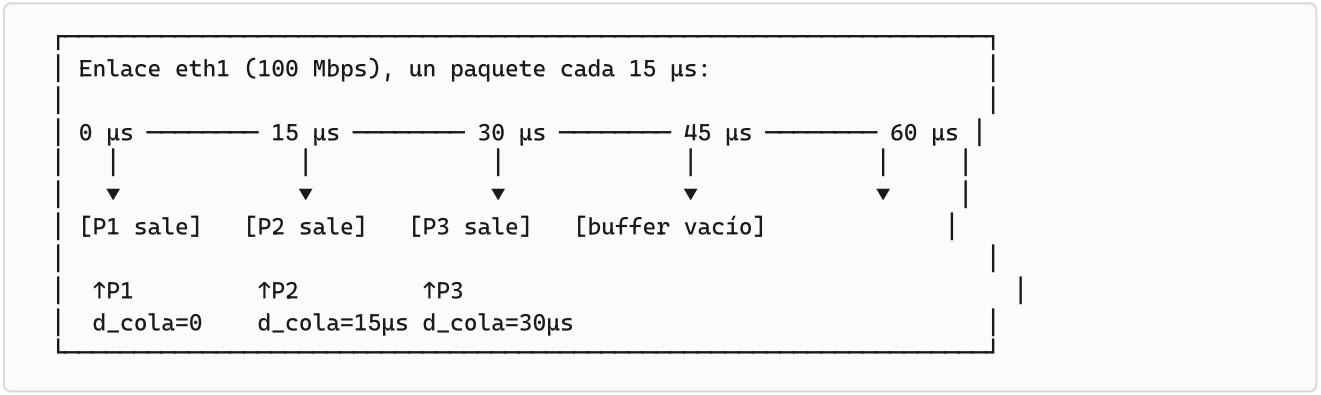
DESCARTADOS = 0
EN COLA     = 3   (los 3 esperan turno)
```

Paso 4 — Respuesta a b): el retardo de cola de cada uno.

El retardo de cola del paquete que está en la posición `k` es el tiempo que tardan los `k-1` paquetes que tiene adelante en salir por el enlace de salida:

```
d_cola(P1) = 0 × (L/R) = 0                    ← es el primero, sale ya
d_cola(P2) = 1 × (1500/100 Mbps) = 15 µs
d_cola(P3) = 2 × (1500/100 Mbps) = 30 µs

(L/R = 1500 bits / 100 000 000 bit/s = 15 µs por paquete)
```



Paso 5 — Respuesta a c): paquetes de 40 bytes.

```
Buffer = 20 000 bytes, paquete = 40 bytes
Capacidad = ⌊20 000 / 40⌋ = 500 paquetes

Llegan 3. Caben todos. DESCARTADOS = 0.

d_cola(P2) = 1 × (40 bytes × 8 / 100 000 000)
           = 1 × 320 / 100 000 000
           = 3,2 µs

d_cola(P3) = 2 × 3,2 µs = 6,4 µs

Comparación:
   con 1500 B: d_cola(P3) = 30 µs
   con   40 B: d_cola(P3) =  6,4 µs      → 4,7 veces MENOS
```

Interpretación del apartado c): con paquetes chicos, **el mismo volumen de tráfico se almacena y se sirve con menos retardo de cola**, porque la granularidad de la cola es fina. Pero el costo es **overhead**: 58 bytes de cabecera por cada 40 bytes de datos = más del 100 % de overhead. Es el mismo compromiso de siempre: **granularidad contra eficiencia**, y es exactamente el análisis de la Pregunta 1b del Examen 2007-12 sobre asignación de recursos.

⚠️ Errores típicos

- **Contar bytes en vez de paquetes en el buffer.** Con 20 000 bytes de buffer entran 13 paquetes de 1500 B, no 13,33. Un paquete a medias no sirve.
- **Calcular `d_cola` como el tiempo total de la cola** y no el tiempo de los paquetes que tiene **delante**. El primer paquete de una cola vacía tiene `d_cola = 0`.
- **Olvidar que el retardo de cola se suma al de transmisión.** El paquete no "sale" en el instante `k·L/R`: empieza a salir después de esperar, y tarda `L/R` más en salir.
- **Confundir el buffer de eth1 con el de eth0.** El eth2 a 10 Gbps drena por eth1 (100 Mbps), así que **eth1 es el cuello de botella** y es donde hay cola. eth0 no tiene tráfico.

2.2 Ejercicio 2 — Utilización de un circuito frente a paquetes con tráfico en ráfagas

Enunciado. Dos aplicaciones comparten un enlace de **1 Mbps**:

- **Aplicación A**: tráfico en **ráfagas**: transmite 40 kbit durante 100 ms, y después **90 000 ms** (90 segundos) en silencio.
- **Aplicación B**: transmite de forma **continua** los 1 024 kbit que le quedan.
- El problema dura **15 minutos** = **900 segundos**.

a) ¿Cuánto dato transmite A en los 15 minutos? ¿Y B? **b)** ¿Cuál es la utilización media del enlace si A va por un **circuito reservado** de 1 Mbps? ¿Y si van por **paquetes**?

Resolución.

a) El volumen de datos de cada una.

```
A: 15 ráfagas (una cada 60 segundos en 900 s)
   15 × 40 kbit = 600 kbit

B: 900 s × 1024 kbit/s = 921 600 kbit
```

b) Con circuito reservado.

```
┌──────────────────────────────────────────────────────────┐
│ CIRCUITO: A recibe 1000 kbit/s reservados para 900 s     │
│                                                          │
│ Datos entregados por el circuito de A: 1000 × 900 = 900 000 kbit  │
│ Datos REALES que A tenía:                       600 kbit │
│                                                          │
│ ⚠️ SE PIERDEN (o se ocupan sin usar) 899 400 kbit = 99,93 %  │
│                                                          │
│ Link utilization por circuito = 900 000 / (1000 × 900) = 100 %  │
│ A (utilización efectiva de A) =   600 / 900 000 = 0,067 %  │
└──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────────┐
│ PAQUETES: A usa el enlace solo en sus ráfagas                │
│                                                              │
│ A transmite 600 kbit en 15 × 100 ms = 1,5 s de uso real      │
│ B transmite 921 600 kbit en 900 s                            │
│                                                              │
│ Link utilization = (600 + 921 600) / (1000 × 900)            │
│                  = 922 200 / 900 000 = 102,5 %   ⚠️ ¡NO LLEGA! │
└──────────────────────────────────────────────────────────────┘
```

Aquí hay que hacer el ajuste honesto. Si los datos que B quiere mandar son 921 600 kbit en 900 s, el enlace de 1 Mbps no alcanza. La solución es que **el enlace real sea de 1,024 Mbps** o que B sea más lento. Supongamos que el enlace es de 1 024 kbps y que el reparto es proporcional:

```
ANÁLISIS CORRECTO con enlace de 1024 kbps y reparto por paquetes

Total deseado en 900 s:
   A:       600 kbit
   B:   921 600 kbit
   ─────────────────
   total:  922 200 kbit

Capacidad del enlace en 900 s:
   1024 kbit/s × 900 s = 921 600 kbit

⚠️ El total pedido (922 200) EXCEDE la capacidad (921 600) en
   600 kbit, que es exactamente lo que A pide.
   ⇒ El enlace está saturado el 100 % del tiempo.

Con PAQUETES, el reparto es el que resulte de la competencia:
   cada uno manda lo que puede, y en los valles de uno
   aprovecha el otro.

Link utilization = 100 % en ambos casos (el enlace está lleno).

LA DIFERENCIA ESTÁ EN QUÉ SE PIERDE:

┌──────────────────────────────────────────────────────────────┐
│ CON CIRCUITO                                                 │
│   1000 kbit/s reservados para A                              │
│   A los usa: 0,6 s de 900 s (0,067 %)                        │
│   Lo que sobra: 999,4 s × 1000 kbit/s = 899 400 kbit DESPERDICIA │
│   → Para que B reciba su 921 600 kbit habría que agrandar el │
│     enlace a 1024 + 1000 = 2024 kbps                         │
│                                                              │
│ CON PAQUETES                                                 │
│   Ningún ancho de banda reservado                            │
│   En los 1,5 s de ráfagas de A, el enlace va al 100 %        │
│   En los 898,5 s restantes, el enlace lo usa B               │
│   → Con 1024 kbps alcanza, sin gastar nada                   │
└──────────────────────────────────────────────────────────────┘
```

El resultado final, que es el que hay que decir:

	Circuito reservado	Paquetes (mejor esfuerzo)
Capacidad necesaria	**2 024 kbps**	**1 024 kbps**
Desperdicio	**899 400 kbit** (99,93 % del circuito)	**0**
Retardo de A durante su ráfaga	**Constante y bajo** (tiene el enlace)	**Competitivo** (puede esperar)
Retardo de B	**Competitivo** (A no le ocupa el ancho, pero sí ocupa el nodo)	**Competitivo**
Retardo de A en su silencio	N/A	N/A

Interpretación — y esto es lo que hace que Internet use paquetes:

- **El circuito da calidad** (A siempre tiene lo suyo) **pero se paga caro**: hace falta el doble de capacidad para el mismo servicio, y el 99,9 % de esa capacidad se paga sin usar.
- **Los paquetes dan eficiencia** (el mismo enlace sirve para las dos aplicaciones al 100 %) **pero no dan calidad**: A compite con B cuando dispara su ráfaga.
- **El compromiso real** es que el enlace tiene que estar **sobrecargado a propósito**: hay que dimensionarlo para el 100 % del tráfico, lo cual significa **más retardo en la cola**. Por eso la utilización de un enlace debe ser ~90 % y no 100 %, aunque eso signifique capacidad ociosa.

2.3 Ejercicio 3 — ¿Por qué los paquetes de tamaño variable no se usan?

Enunciado. Se propone una variante de la conmutación por paquetes donde **cada paquete puede tener un tamaño cualquiera, entre 1 byte y 1500 bytes**, decided por la aplicación. **Justificá por qué esa variante es mala**, con argumentos concretos.

Resolución — los dos argumentos que hay que dar.

Argumento 1 — Injusticia (*unfairness*) y retardo extremo.

```
Enlace de 100 Mbps. Llegan paquetes de tamaño variable.

Llega un paquete de 1500 bytes primero. Tarda:
    1500/100 Mbps = 15 µs en ser transmitido

Justo después llegan 10 paquetes de 100 bytes. El primero tiene que
esperar los 15 µs del grande; el segundo, 30 µs; el último, 150 µs.

┌──────────────────────────────────────────────────────────┐
│ t (µs)   0      15      30      45      60  ...  150     │
│          |       |       |       |       |       |      │
│ enlace   [  1500 B  ][100][100][100][100][100]...       │
│                                                          │
│ RTT del paquete grande (en el sentido de la cola): 15 µs │
│ RTT del último paquete chico:               150 µs       │
│                                                          │
│ ⚠️ 10 veces peor. Y el tamaño del paquete grande es un factor │
│    que el operador NO controla: lo elige la aplicación.  │
└──────────────────────────────────────────────────────────┘
```

Consecuencias:

- **El retardo de un paquete depende del comportamiento de otros**, no del suyo. Un paquete de 100 bytes puede tener un retardo 1000 veces mayor que otro de 100 bytes si justo delante hay uno de 1500 bytes.
- **No hay garantía de retardo.** Y en un sistema en tiempo real, **no tener garantía es lo mismo que no servir**.
- **El culpable no es la red**: es la aplicación que mandó el paquete grande. La red solo obedece. Pero la red es la que carga con la culpa.

Argumento 2 — El buffer tiene que ser adaptativo, y eso rompe el store-and-forward acotado.

```
Con paquetes de tamaño FIJO L, el router puede saber EXACTAMENTE
cuántos packets caben:

    capacidad = ⌊B / L⌋     (determinista, conocido, acotado)

Con paquetes de tamaño VARIABLE (1 a 1500 bytes), el router no
puede saberlo:

    Un paquete de 1500 bytes ocupa 1500.
    Un paquete de 1 byte ocupa 1.
    La MISMA cola de B = 20 000 bytes contiene:
        13 paquetes de 1500 bytes       →  13 paquetes
     o 20 000 paquetes de 1 byte        →  20 000 paquetes

    ⚠️ El retardo de cola varía en un factor de 1 500 según
       qué paquetes hay en la cola. Y el router no puede predecirlo.
```

Los argumentos secundarios:

3. **Overhead ilegible.** No podés calcular la eficiencia sin saber el tamaño. Con `L` variable, la eficiencia `L/(L+58)` **varía por paquete**, y el promedio depende de la distribución de tamaños, que depende de las aplicaciones.
4. **Predicción imposible para el sistema de transporte.** Un AQM (*Active Queue Management*) como **CoDel** mide cuánto lleva en la cola un paquete. Con tamaños variables, la medida es injusta.
5. **Multiplexación y prioridad.** Para priorizar por flujo (DSCP en la cabecera IP), el router debería tratar igual un paquete de 1 byte que uno de 1500 bytes **en bytes**, no en cantidad. Con tamaño variable hay que definir la unidad de medida, y cualquier elección genera repartos injustos.

Conclusión para el examen:

```
El tamaño FIJO del paquete no es un detalle de implementación:
es lo que hace POSIBLES tres cosas:
  1. el store-and-forward con buffer FINITO y conocido
  2. una GARANTÍA (o al menos un cálculo) del retardo de cola
  3. la EQUIDAD entre paquetes
Por eso el MTU existe, y por eso existe el MSS.
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2007-12, Pregunta 3 (10 puntos) — las tres técnicas con análisis comparativo

> **Fuente:** Examen `eirc0712` / `sirc0712` — 27 de diciembre de 2007, Pregunta 3 (10 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0712.pdf

Enunciado (texto del examen)

> Explique en que consiste la conmutación de circuitos, conmutación de paquetes y conmutación de mensajes. Realice un análisis comparativo.

Sobre la solución oficial

> ⚠️ **La solución oficial de esta pregunta NO está publicada.** En `sirc0712.txt` el enunciado de la Pregunta 3 aparece (líneas 29-31) y después el documento pasa directamente a "Problemas Prácticos" y a la solución del Problema 1. **No hay respuesta de la cátedra para esta pregunta.** Todo lo que sigue es una **solución propia**, desarrollada con el esquema de Tanenbaum y con la aritmética del §1.4. Lo marco como tal.

Resolución *(solución propia, para 10 puntos)*

a) Conmutación de circuitos.

```
Se ESTABLECE un circuito entre los dos extremos antes de
transferir datos, y en cada nodo del camino se RESERVAN recursos
(un ancho de banda fijo, normalmente un *time slot*).

La fase de establecimiento ocurre FUERA DE BANDA: hay un canal de
control separado que lleva las señales de ocupado/libre. Es un
diálogo entre los nodos que reservan y liberan.

Una vez establecido:
   · los datos viajan SIEMPRE por el mismo camino
   · el retardo es CONSTANTE (no hay cola, porque el ancho de
     banda está reservado)
   · si un nodo del camino falla, se rompe la comunicación: no hay
     ruta alternativa
   · los recursos quedan ociosos durante las pausas del tráfico,
     y se pagan igual

Ejemplo canónico: la red telefónica clásica. Llamar establish un
camino de 64 kbps reservado a través de todos los nodos
intermedios.
```

b) Conmutación de paquetes.

```
El mensaje se DIVIDE en paquetes de tamaño FIJO o acotado. Cada
paquete se reenvía de manera INDEPENDIENTE, con store-and-forward,
y la decisión de por dónde va se toma PARA CADA PAQUETE, mirando la
dirección de destino.

Consecuencias:
   · NO hay establecimiento: el primer paquete sale ya
   · NO se reserva ancho de banda: es "mejor esfuerzo" (best effort)
   · el retardo es VARIABLE, y depende de la cola de cada router
   · dos paquetes del mismo mensaje pueden ir por caminos distintos
     → pueden llegar desordenados o perderse
   · los routers son STATELESS: no guardan información del flujo

Es lo que usa Internet.
```

c) Conmutación por mensajes.

```
El mensaje COMPLETO, de tamaño arbitrario y potencialmente enorme,
se reenvía de nodo en nodo. Cada nodo lo recibe ENTERO, lo guarda
en un buffer, y ESPERA a poder transmitirlo completo.

Es store-and-forward SIN FRAGMENTAR, sin cota superior de tamaño.

Consecuencias:
   · NO hay establecimiento
   · el retardo es muy grande: el mensaje entero tiene que
     atravesar cada enlace antes de que el siguiente nodo pueda
     empezar a transmitir
   · BLOQUEO DE CABEZA: un mensaje grande bloquea a todos los
     pequeños que están detrás en el buffer
   · el buffer tiene que poder contener el mensaje más grande
     que se pueda recibir, lo que es inviable en un router

Es el antecesor histórico de la conmutación por paquetes: si se
acota el tamaño del mensaje, se convierte en conmutación por
paquetes.
```

d) El análisis comparativo. La tabla del §1.6 completa, reproducida:

Característica	Circuitos	Paquetes	Mensajes
¿Hay conexión?	Sí, se establece	No (datagramas)	No
Unidad de datos	Flujo continuo	Paquete de tamaño acotado	Mensaje completo
¿Se reservan recursos?	**Sí**	No	No
Retardo	**Constante**	Variable	Variable y muy grande
¿Puede cambiar el camino?	No	Sí, por paquete	Sí, por mensaje
¿Llegan en orden?	Siempre	Puede que no	Puede que no
Establecimiento	100 ms a 2 s	Ninguno	Ninguno
Tamaño del buffer	No hace falta	Acotado y conocido	Debe ser enorme
¿Escala?	No	**Sí**	No
Uso real	Teléfono, MPLS (LSP)	**Internet**	Prácticamente ninguno

La conclusión del análisis comparativo, que es lo que la pregunta pide cerrar:

```
Las tres sirven para COMPARTIR un medio. Lo que cambia es
QUÉ se comparte:

· Circuitos:  el ANCHO DE BANDA.   Reparto fijo y estático.
· Paquetes:   el TIEMPO.           Reparto dinámico por oportunidad.
· Mensajes:   el TIEMPO, a bloque. Reparto dinámico pero bloqueado.

El circuito da CALIDAD a costa de EFICIENCIA.
Los paquetes dan EFICIENCIA a costa de CALIDAD.
Los mensajes no dan ninguna de las dos.

La red eligió paquetes porque Internet tiene una mezcla de tráfico
(ráfagas de archivos, ráfagas de voz, video constante) donde la
eficiencia importa más que la calidad, y porque la calidad se
puede COMPRAR después en un DiffServ (Clase 45), mientras que la
eficiencia no se puede comprar.
```

3.2 ⭐ Examen 2009-02, Pregunta 4 (9 puntos) — datagramas vs. circuitos virtuales

> **Fuente:** Examen `sirc0902` — 2 de febrero de 2009, Pregunta 4 (9 puntos). **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0902.pdf

Enunciado (texto del examen)

> **a)** Describa la conmutación de paquetes basada en: **i.** Datagramas **ii.** Circuitos virtuales.
>
> **b)** Explique ventajas y desventajas de cada opción desde el punto de vista de la **asignación de recursos en la red**.

Solución oficial

> **c) Describa la conmutación de paquetes basada en: i.** Datagramas **ii.** Circuitos virtuales. *(Sin desarrollo: el PDF deja el apartado en blanco.)*
>
> **d) Explique ventajas y desventajas de cada opción desde el punto de vista de la asignación de recursos en la red.**
>
> Básicamente las redes de datagramas son "de mejor esfuerzo", es decir que consumen los recursos disponibles, y en caso de congestión degradan el servicio por igual a todos los usuarios; aprovechan bien el multiplexado estadístico del tráfico en la red. Las redes de VC ofrecen garantía de QoS basadas en la reserva de recursos, y por lo tanto protegen a los usuarios de las situaciones de congestión, a costo de disponer de más recursos o rechazar conexiones (control de admisión).
>
> ⚠️ **Dos errores de la solución oficial que hay que notar:** (a) los apartados de la solución están rotulados **c)** y **d)** cuando el enunciado dice **a)** y **b)**: es un error de maquetación de la cátedra. (b) El apartado **a)** —la descripción de las dos técnicas— **quedó sin desarrollar**. Lo desarrollo abajo.

Análisis de la solución oficial

Lo que está muy bien:

1. **El resumen del apartado b) es una de las mejores frases de todo el material oficial**: "*aprovechan bien el multiplexado estadístico del tráfico en la red*". Es exactamente el argumento de por qué Internet es de datagramas, en una frase.
2. **Nombra los dos costos de los circuitos virtuales**: o **disponés de más recursos**, o **rechazás conexiones**. El segundo es el **control de admisión** (*admission control*), que es el término técnico correcto y que casi ningún alumno conoce. Lo nombró.
3. **Distingue degradación uniforme vs. garantía**: en datagramas, la congestión "degrada el servicio por igual a todos los usuarios"; en VC, "protege a los usuarios de las situaciones de congestión". Esa es la diferencia observable.

Lo que falta:

1. **El apartado a) entero.** Ni datagramas ni circuitos virtuales están descritos. Hay que hacerlo.
2. **No dice qué hace el router en cada caso.** En datagramas, el router busca por la IP destino; en VC, por la etiqueta. Esa es la diferencia mecánica.
3. **No menciona el estado.** Un router de datagramas es **stateless**; uno de VC mantiene una **tabla de VC** (estado). Ese estado es lo que permite la QoS y lo que hace que no escale.
4. **No menciona la etiqueta.** El VC se identifica por una etiqueta en la cabecera, no por la IP de destino.

Parte a) — las dos técnicas *(desarrollo propio, la oficial lo dejó vacío)*

i. Datagramas

```
Cada paquete es INDEPENDIENTE y lleva la DIRECCIÓN DE DESTINO
COMPLETA (la IP) en su cabecera. No hay conexión, no hay estado, no
hay negociación previa.

┌──────────────────────────────────────────────────────────────┐
│  Paquete 1:  destino = 203.0.113.9, TTL = 64                 │
│  Paquete 2:  destino = 203.0.113.9, TTL = 63                 │
│  Paquete 3:  destino = 203.0.113.9, TTL = 64                 │
│                    ↑ 3 paquetes, 3 cabeceras completas       │
└──────────────────────────────────────────────────────────────┘

El ROUTER hace, por cada paquete:
   1. lee la IP destino
   2. longest prefix match en su tabla de reenvío
   3. reenvía por la interfaz que le corresponde

· No guarda NADA entre paquetes. Es STATELESS.
· Dos paquetes con el mismo destino pueden tomar caminos distintos.
· Si cambia la topología, el siguiente paquete va por el camino nuevo.
· Los paquetes pueden perderse, duplicarse o llegar desordenados.
· ES EL MODELO DE INTERNET (IP, RFC 791; y RFC 8200 para IPv6).
```

ii. Circuitos virtuales

```
Se ESTABLECE un camino virtual entre los extremos antes de mandar
datos. Cada paquete lleva una ETIQUETA que identifica el camino, y
los routers reenvían MIRANDO LA ETIQUETA, sin consultar la IP de
destino.

┌──────────────────────────────────────────────────────────┐
│  Paquete 1:  etiqueta = 7, VC = (H1 → H2)                │
│  Paquete 2:  etiqueta = 7, VC = (H1 → H2)                │
│  Paquete 3:  etiqueta = 7, VC = (H1 → H2)                │
│                   ↑ 3 paquetes, la MISMA etiqueta        │
└──────────────────────────────────────────────────────────┘

El ROUTER hace:
   1. lee la ETIQUETA de la cabecera MPLS
   2. busca en su TABLA DE VCS (que es un array, no un longest
      prefix match)
   3. cambia la etiqueta por la de salida y reenvía

· El router mantiene ESTADO: la tabla de VCs y los contadores.
· Todos los paquetes del mismo VC van por el MISMO camino.
· El orden de llegada se preserva (mismo camino, mismo buffer).
· Los VC se pueden AGRUPAR en un mismo enlace físico.
· Implementaciones: ATM (AAL5), Frame Relay, MPLS (etiqueta de
  20 bits, LSP unidireccional), y VXLAN en el datacenter.
```

La comparación mecánica, que es la que faltaba:

	Datagramas	**Circuitos virtuales**
Qué mira el router	La **IP destino**	La **etiqueta** de VC
Cómo busca	**Longest prefix match** en la tabla de ruteo	**Índice directo** en una tabla de VCs
Coste de la decisión	Acceso a una TCAM (lento)	Un índice (rápido)
Estado en el router	**Ninguno**	**Por VC**: contadores, cola, QoS
Camino por paquete	Puede cambiar	**Fijo**
Sobrevive a un cambio de topología	**Sí**: el siguiente paquete va por donde sea	**No**: hay que **reestablecer** el VC
Cuentas	Sobrecarga de la cabecera IP	Sobrecarga de la etiqueta MPLS (4 bytes)

La observación clave sobre la velocidad: el VC es **más rápido de conmutar** porque la decisión es un índice en un array, no una búsqueda de prefijo. **Ese es el motivo técnico de MPLS**: en un backbone a 100 Gbps, la diferencia entre un índice y una TCAM es la diferencia entre 40 ns y 3000 ns por paquete. Pero ese ganho viene **precisamente del estado**, y el estado es lo que hace que no escale.

Parte b) — ventajas y desventajas desde la asignación de recursos *(desarrollo de la solución oficial)*

La oficial en dos frases. La desarrollo con los dos casos y las dos preguntas que hay que responder.

Datagramas — desde el punto de vista de la asignación de recursos:

```
ASIGNACIÓN DE RECURSOS: NINGUNA, IMPLÍCITA, POR OPORTUNIDAD.

· Es "mejor esfuerzo" (best effort): cada paquete usa los recursos
  que encuentra libres en ese instante, sin preguntar y sin
  reservar.
· MULTIPLEXADO ESTADÍSTICO: los picos de un flujo llenan los
  valles de otro. Es la razón por la que un único enlace de
  100 Mbps sirve para 1000 usuarios que cada uno usan 200 kbps el
  10 % del tiempo.

  límite  ──────────────────────────────────────────────────►
  1000 flujos × 200 kbps × 10 % = 200 kbps de MEDIA
  enlace de 100 Mbps = 100 000 kbps
  factor de multiplexado = 100 000 / 200 = 500×

· En caso de congestión, TODOS los usuarios se ven afectados
  IGUALMENTE. No hay manera de proteger a nadie.

VENTAJAS
   · Eficiencia máximo del enlace
   · No necesita estado en los routers → escala a millones de flujos
   · Sobrevive a los cambios de topología
   · No hay establecimiento ni control de admisión

DESVENTAJAS
   · Sin garantía de retardo ni de ancho de banda
   · La congestión degrada a todos por igual
   · Para dar QoS hay que marcar los paquetes (DiffServ) y eso
     obliga a tener ESTADO por clase, que es menos que por flujo
     pero más que cero
```

Circuitos virtuales — desde el punto de vista de la asignación de recursos:

```
ASIGNACIÓN DE RECURSOS: EXPLÍCITA, POR RESERVA, CON CONTROL
DE ADMISIÓN.

· Al establecer el VC se RESERVAN recursos en cada salto:
  ancho de banda y/o buffer.
· CONTROL DE ADMISIÓN (*admission control*): si no hay recursos
  suficientes, NO SE ACEPTA el VC. Se rechaza la conexión. Esto es
  fundamental: es lo que hace que las garantías sean reales.

  enlaces disponibles: 3 × 64 kbps
  VC pedido 1:  64 kbps  → ok, quedan 2 × 64
  VC pedido 2:  64 kbps  → ok, quedan 1 × 64
  VC pedido 3:  64 kbps  → ok, quedan 0
  VC pedido 4:  64 kbps  → RECHAZADO (control de admisión)

· AISLAMIENTO: la congestión de un VC no afecta a los demás, porque
  los recursos están reservados y separados.

VENTAJAS
   · GARANTÍA de retardo y de ancho de banda
   · AISLAMIENTO entre flujos: uno no puede arruinar al otro
   · Preserva el ORDEN (mismo camino)
   · El flujo es predecible: no hay picos de retardo

DESVENTAJAS
   · COSTE: hay que tener más recursos disponibles de los que se
     usan, o aceptar conexiones rechazadas
   · ESTADO por VC en cada router: no escala a millones de flujos
     (es el motivo por el que IntServ, RFC 2211, no se despliega)
   · Si cae un nodo, se rompe el VC y hay que reestablecerlo
   · La reserva ociosa: se paga y no se usa
```

La frase de síntesis, que es la conclusión de la pregunta:

```
┌──────────────────────────────────────────────────────────────┐
│  Datagramas: asignación IMPLÍCITA, por oportunidad.          │
│  → Eficiencia máxima, cero garantías, cero estado.           │
│                                                              │
│  Circuitos virtuales: asignación EXPLÍCITA, por reserva.     │
│  → Garantías reales, pero con coste en recursos y en estado. │
│                                                              │
│  Y la clave histórica: IntServ (reserva por flujo) NO escaló.│
│  DiffServ (marcar por clase, sin reserva) escaló. Porque en  │
│  Internet la cantidad de flujos es tan grande que el estado  │
│  por flujo es insostenible, mientras que el estado por clase │
│  (un número pequeño de colas) es viable.                     │
└──────────────────────────────────────────────────────────────┘
```

3.3 Referencia — Examen 2009-07 P2 (10 puntos), sobre el mismo terreno

> **Enunciado:** *"Un problema de enrutamiento donde la métrica es la distancia en número de saltos (hops): ¿es la mejor métrica posible? Justifique."*

Por qué lo pongo acá, aunque sea de otra clase: el enunciado es sobre enrutamiento (Clase 33), pero **el fondo es la conmutación**. La respuesta oficial dice, textual:

> *"Si se busca una utilización eficiente de los recursos y/o satisfacer requerimientos de calidad de servicio, la métrica de hops puede ser inconveniente, ya que señalará como 'mejor' el camino más corto, que no necesariamente será el que tenga mejores atributos de calidad (retardo extremo a extremo, pérdidas, throughput) y/o menos congestión."*

La conexión con esta clase: el **número de hops es la métrica que corresponde a los circuitos virtuales**, porque mide el camino fijo, no la calidad. En **datagramas** uno podría medir la calidad real (retardo, ancho de banda) y elegir por eso; en **circuito virtual** el camino ya está fijado y solo queda medirlo para calcular el retardo. Y la objeción de la oficial es que **el camino más corto en hops no es el más corto en retardo**: 3 saltos por un enlace satelital de 600 ms son peores que 8 saltos por fibra de 2 ms cada uno.

PARTE 4 — CIERRE

📌 Resumen

1. **El criterio único** que distingue a las tres: **¿cuándo se decide el camino** (antes / por cada unidad de datos).
2. **Circuitos**: se establece un camino y **se reservan recursos**. Retardo **constante**, orden garantizado, si cae un nodo **se rompe**, y **los recursos ociosos se pagan**. La red telefónica clásica es el ejemplo.
3. **Mensajes**: el mensaje **completo** se reenvía entero, con store-and-forward. Retardo enorme, **bloqueo de cabeza**, buffers que tienen que ser enormes. Es el **antecedor** de los paquetes.
4. **Paquetes**: el mensaje se **divide en paquetes acotados** y cada uno se reenvía por separado. **Es lo que usa Internet.** Compromiso entre mensajes y circuitos: **mensaje de tamaño de mensaje pero store-and-forward de paquete pequeño**.
5. **Store-and-forward** es obligatorio porque el router tiene **buffers finitos** y los enlaces de entrada y salida tienen **velocidades distintas**. Si reenvía bit a bit, necesita memoria infinita.
6. **El pipeline** es lo que gana los paquetes: con `N` paquetes y `n` enlaces, el último tarda `(N + n − 1)·L/R`. Los mensajes tardan `N·n·L/R`; el circuito, `N·L/R` pero con `100 ms a 2 s` de establecimiento.
7. **El router con buffers llenándose**: el retardo de cola del paquete en posición `k` es `k·L/R`, y si el buffer se llena **se descarta** (pérdida). El buffer es una cola, y toda cola tiene un límite.
8. **Comparación clave**: circuitos = calidad a costa de eficiencia; paquetes = eficiencia a costa de calidad; mensajes = ni una ni otra.
9. **Datagramas vs. circuitos virtuales**: el router mira **la IP destino** (longest prefix match, stateless) o **la etiqueta** (índice, stateful). El VC es **más rápido** pero **no escala** por el estado.
10. **Por qué Internet usa paquetes**: sin establecimiento, sin reserva ociosa, se puede reenrutar, y el núcleo es **stateless**. **IntServ no escaló**; DiffServ ganó.

✅ Autoevaluación

1. ¿Por qué un router no puede reenviar los datos bit a bit, como el *cut-through forwarding* de los switches rápidos?

▶ Respuesta

2. Un mensaje de 30 000 bits va por 3 enlaces de 2000 bit/s. ¿Cuánto tarda con conmutación de mensajes, con paquetes de 5000 bits, y con circuito?

▶ Respuesta

3. ¿Qué es "multiplexado estadístico" y por qué es la ventaja de los datagramas?

▶ Respuesta

4. ¿Por qué un circuito virtual es más rápido de conmutar que un datagrama, si los circuits virtuales requieren más estado?

▶ **Respuesta**

5. Explique por qué el tamaño del buffer de un router no protege contra la pérdida.

▶ **Respuesta**

6. ¿Qué es control de admisión y por qué es indispensable en los circuitos virtuales?

▶ **Respuesta**

7. Un mensaje de 8000 bits se divide en paquetes de 1000 bits y atraviesa 5 enlaces de 500 bit/s. ¿Cuánto tarda el último paquete?

▶ **Respuesta**

8. ¿Por qué la conmutación por mensajes "no se usa"? Y sin embargo existen los *mail relay*.

▶ **Respuesta**

🔗 Referencias

Exámenes - Examen `sirc0712` (27-dic-2007) — P3 (10 pts): las tres técnicas con análisis comparativo — **la cátedra NO publicó la solución oficial** · P2 (10 pts): leaky bucket y token bucket - Examen `sirc0902` (2-feb-2009) — P4 (9 pts): datagramas vs. circuitos virtuales, y la asignación de recursos · Problema 1 (30 pts): análisis de enlaces paralelos en link-state - Examen `sirc1103` (21-feb-2011) — P5 (10 pts): retardo con store-and-forward y con reenvío bit a bit → **Clase 6** - Examen `sirc0907` (2-jul-2009) — P2 (10 pts): distancia en hops como métrica, ¿es la mejor? - Examen `sirc0608` (5-ago-2006) — P3 (10 pts): distance-vector, ventajas y desventajas · Problema 1b: Dijkstra oscilando entre enlaces paralelos

Bibliografía - Tanenbaum & Wetherall, *Redes de computadoras*, 5ª ed., **cap. 1 §1.3** "La red de conmutación de paquetes" (pág. ~10 a ~20): las tres técnicas con el diagrama de los 4 nodos - Tanenbaum & Wetherall, **cap. 1 §1.2** "Tipos de red" — la clasificación PAN/LAN/WAN - Tanenbaum & Wetherall, **cap. 1 §1.6** — el modelo TCP/IP y por qué Internet usa datagramas - Kurose & Ross, *Computer Networking*, cap. 1 §1.4 "What's Inside a Router" — **store-and-forward** con los números y el caso del "4 enlaces a 10 Mbps y uno a 11 Mbps" - Kurose & Ross, cap. 1 §1.2 "The Network Edge" — conmutación de circuitos frente a conmutación de paquetes, con los números de los *timeslots*

RFCs - `RFC 791` — *Internet Protocol* (1981): el modelo de datagramas, sin conexión y sin estado - `RFC 8200` — *Internet Protocol, Version 6* (2017, *Internet Standard*): el mismo modelo de datagramas, 37 años después - `RFC 2737` — *Guidelines for Management of IP-Based Intranets* (1999): las restricciones de *ingress filtering* que acotan el reenvío y el flooding - `RFC 3031` — *Multiprotocol Label Switching Architecture* (2001): el circuito virtual moderno (`RFC 3032` pila de etiquetas, `RFC 4364` MPLS VPN) - `RFC 8402` — *Segment Routing Architecture* (2018), `RFC 8660` (SR-MPLS), `RFC 8986` **(SRv6, 2022)**: el sucesor

que codifica el circuito virtual en la cabecera IPv6 - `RFC 7348` — *VXLAN* (2014): el circuito virtual **de capa 2** que reemplazó a MPLS en el datacenter, encapsulado sobre UDP/4789 - `RFC 2211` — *Integrated Services* (1997): el modelo de reserva por flujo que **no escaló** · `RFC 2474` / `RFC 2475` (DiffServ): el modelo por clases que **sí escaló** - `RFC 9438` — *CUBIC* (2023, *Standards Track*): el control de congestión por AIMD que asume conmutación de paquetes

Clase siguiente: 5 — El retardo en las redes de paquetes